

这版不是只覆盖你已经发过的考试题，而是按 lecture notes 的主线。把可能被改编成计算题、证明题、代码解释题的知识点系统整理。每一节包含：

1. 这部分在 note 里讲什么
2. 考试已经怎么考过
3. 同一部分还可能怎么考
4. 计算 / 证明模板

0. 总复习策略

这门课的理论考试题通常不是让你背一大段定义，而是让你做四类事情：

1. **shape** / 参数量 / **forward** 计算
 - PyTorch Linear, Conv2d, attention, RNN
2. 公式代入计算
 - softmax, attention, convolution, pooling, upsampling, deconvolution
3. 链式法则 / **backprop** / **computation graph**
 - data flow graph, partial derivatives, online softmax
4. 概率模型证明
 - ELBO、softmax over-parametrization、DDIM deterministic update

考试时优先识别题型：

看到 softmax -> exp / sum exp，注意平移不变性

看到 graph -> 路径乘积相加

看到 PyTorch -> shape、参数量、zero_grad/backward/step

看到 conv -> kernel、stride、padding、dilation、channel

看到 RNN -> 展开 h_t

看到 ELBO -> 乘 q/q，用 Jensen，拆 p(x,z)

看到 DDIM -> sigma=0，x0 estimate 代入

看到 attention -> QK^T / sqrt(d) -> softmax -> V

看到 online algo -> loop invariant

1. Tensor / Linear Algebra / PyTorch Shape

1.1 Note 里讲什么

深度学习中有所有数据基本都是 tensor：

- vector：一维数组
- matrix：二维数组
- tensor：任意维数组

图片在 PyTorch 中通常是：(C, H, W)

batch 之后：(B, C, H, W)

普通向量 batch：(B, N)

1.2 可能考什么

■ 题型 A：矩阵乘法 **shape** 如果：

$$A \in \mathbb{R}^{m \times n}, \quad x \in \mathbb{R}^n$$

则：

$$Ax \in \mathbb{R}^m$$

如果 batch 输入：

$$X \in \mathbb{R}^{B \times n}$$

则 Linear 层输出：

$$Y \in \mathbb{R}^{B \times m}$$

PyTorch 中：
nn.Linear(in_features=n, out_features=m)

输入：(B, n)

输出：(B, m)

■ 题型 B：transpose / reshape / flatten 区别 给：

$$A = \begin{bmatrix} 8 & 5 & 1 \\ 3 & 6 & 2 \end{bmatrix}$$

A.T 是：

$$A^\top = \begin{bmatrix} 8 & 3 \\ 5 & 6 \\ 1 & 2 \end{bmatrix}$$

但 reshape(3,2) 不是 transpose，它是按内存顺序重排：

$$\begin{bmatrix} 8 & 5 \\ 1 & 3 \\ 6 & 2 \end{bmatrix}$$

flatten()：

$$[8, 5, 1, 3, 6, 2]$$

■ 题型 C：broadcasting 如果：
x.shape = (B, N)
b.shape = (N,) 则：
x + b 会把 b 自动复制到 batch 维度：
(B, N) + (N,) -> (B, N)
常理解释：
bias b 是对 batch 中每个样本都加一次

2. Linear / Affine Function 与偏导

2.1 Note 里讲什么

神经网络最基本的层是：

$$y = Ax + b$$

单个 scalar output 时：

$$f(x) = a^\top x + b$$

2.2 已考题：partial derivatives

题目：

$$f(x; a, b) = a^\top x + b$$

其中：

$$a, x \in \mathbb{R}^n, \quad b \in \mathbb{R}$$

求：

$$\frac{\partial f}{\partial x}, \quad \frac{\partial f}{\partial a}, \quad \frac{\partial f}{\partial b}$$

展开：

$$f = a_1 x_1 + \cdots + a_n x_n + b$$

所以：

$$\frac{\partial f}{\partial x_i} = a_i$$

$$\frac{\partial f}{\partial a_i} = x_i$$

$$\frac{\partial f}{\partial b} = 1$$

答案：

$$\frac{\partial f}{\partial x} = a, \quad \frac{\partial f}{\partial a} = x, \quad \frac{\partial f}{\partial b} = 1$$

2.3 还可能怎么考

■ 可能题：matrix affine layer 的 gradient shape 如果：

$$y = Wx + b$$

其中：

$$W \in \mathbb{R}^{m \times n}, \quad x \in \mathbb{R}^n, \quad b \in \mathbb{R}^m$$

则：

$$y \in \mathbb{R}^m$$

如果 loss L 对 y 的梯度是：

$$g = \frac{\partial L}{\partial y} \in \mathbb{R}^m$$

则常见 backprop 结果：

$$\frac{\partial L}{\partial x} = W^\top g$$

$$\frac{\partial L}{\partial W} = g x^\top$$

$$\frac{\partial L}{\partial b} = g$$

batch 情况下，对 batch 求和或求平均取决于 loss reduction。

3. Logistic Regression / Sigmoid / Regularization / Temperature

3.1 Note 里讲什么

Binary logistic regression：

$$P(y = 1 \mid x) = \sigma(a^\top x + b)$$

其中：

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

性质：

$$\sigma(z) \geq 0.5 \iff z \geq 0$$

所以分类边界是：

$$a^\top x + b = 0$$

3.2 可能考什么

■ 题型 A：给 a, b, x ，计算概率 例如：

$$a = (1, 2), \quad x = (3, -1), \quad b = 0.5$$

先算 logit：

$$z = a^\top x + b = 1 \cdot 3 + 2 \cdot (-1) + 0.5 = 1.5$$

概率：

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-1.5}}$$

如果问分类结果，因为 $z = 1.5 > 0$ ，预测 positive。

■ 题型 B：regularization / weight decay 带 L2 regularization：

$$L_{\text{reg}} = L + \frac{\lambda}{2} \|a\|^2$$

梯度：

$$\nabla_a L_{\text{reg}} = \nabla_a L + \lambda a$$

只看 regularization 项时，gradient descent：

$$a \leftarrow a - \eta \lambda a = (1 - \eta \lambda) a$$

所以 weight decay 会把权重推向 0，使 sigmoid 曲线变平，降低过度自信。

■ 题型 C：temperature scaling

$$\sigma\left(\frac{z}{\tau}\right)$$

- $\tau > 1$: 曲线更平，预测更不自信
- $\tau < 1$: 曲线更陡，预测更自信

softmax 也有类似 temperature：

$$\text{softmax}(z/\tau)$$

4. MLP / Activation / Universal Approximation

4.1 Note 里讲什么

MLP 是 affine layer 和 activation 的组合：

$$z_1 = Ax + b$$

$$h = \sigma(z_1)$$

$$y = c^\top h + d$$

两层 perceptron：

$$y = \sigma(c^\top \sigma(Ax + b) + d)$$

4.2 已考关联：PyTorch MLP 代码

你给的代码：
nn.Linear(4, 8)
nn.ReLU()
nn.Linear(8, 2)
数学上是：

$$h = \text{ReLU}(W_1 x + b_1)$$

$$o = W_2 h + b_2$$

其中：

$$W_1 \in \mathbb{R}^{8 \times 4}$$

$$b_1 \in \mathbb{R}^8$$

$$W_2 \in \mathbb{R}^{2 \times 8}$$

$$b_2 \in \mathbb{R}^2$$

4.3 还可能怎么考

■ 题型 A：activation 输出 ReLU：

$$\text{ReLU}(z) = \max(0, z)$$

如果：

$$z = (-2, 0, 3)$$

则：

$$\text{ReLU}(z) = (0, 0, 3)$$

Sigmoid：

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Tanh：

$$\tanh(z) = \frac{e^{2z} - 1}{e^{2z} + 1}$$

■ 题型 B：为什么 MLP 能解决 XOR Logistic regression 只有一个线性边界，不能解决 XOR。MLP 第一层可以构造多个线性边界，相当于学习新 feature；第二层再对新 feature 分类。

关键词：
hidden layer creates new representation
non-linearity makes composition non-linear

■ 题型 C：Universal Approximation Theorem 结论：

只要 hidden units 足够多，并且 activation 合理，单 hidden-layer MLP 可以近似任意连续函数。

考试可能不是让你严格证明，而是问意义：
MLP 的表达能力很强，但定理不保证容易训练，也不保证泛化好。

5. Softmax / Cross Entropy / Multi-class Logistic

5.1 Softmax 计算规则

对于 logits：

$$z = (z_1, \dots, z_K)$$

softmax：

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

5.2 已考关联：attention 中的 softmax

如果：

$$z = \left[\frac{2}{\sqrt{3}}, 0 \right]$$

则：

$$\text{softmax}(z) = \left[\frac{e^{2/\sqrt{3}}}{e^{2/\sqrt{3}} + 1}, \frac{1}{e^{2/\sqrt{3}} + 1} \right]$$

5.3 Cross entropy

如果 logits 是：

$$o = (o_1, \dots, o_K)$$

target class 是 y ，则：

$$L = -\log \frac{e^{o_y}}{\sum_j e^{o_j}}$$

PyTorch：
nn.CrossEntropyLoss()
输入是 logits，不需要先 softmax。

5.4 已考证 明：multi-class logistic over-parametrized

多分类 logistic：

$$P(y = k \mid x) = \frac{e^{a_k^\top x + b_k}}{\sum_j e^{a_j^\top x + b_j}}$$

构造：

$$c_k = a_k - a_K$$

$$d_k = b_k - b_K$$

则新 score：

$$c_k^\top x + d_k = a_k^\top x + b_k - (a_K^\top x + b_K)$$

也就是所有 class 的 score 都减去同一个东西：

$$s_K = a_K^\top x + b_K$$

softmax 不变：

$$\frac{e^{s_k - s_K}}{\sum_j e^{s_j - s_K}} = \frac{e^{-s_K} e^{s_k}}{e^{-s_K} \sum_j e^{s_j}} = \frac{e^{s_k}}{\sum_j e^{s_j}}$$

结论：
参数不唯一。
同一个预测分布可以由多组参数表示。
可以把第 K 类作为 reference class。

5.5 还可能怎么考

■ 题型 A：softmax 平移不变性 证明：

$$\text{softmax}(z) = \text{softmax}(z + \mathbf{c}1)$$

因为：

$$\frac{e^{z_k + c}}{\sum_j e^{z_j + c}} = \frac{e^c e^{z_k}}{e^c \sum_j e^{z_j}} = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

■ 题型 B：stable softmax 为了避免 e^z overflow，计算：

$$p_k = \frac{e^{z_k - z_{\max}}}{\sum_j e^{z_j - z_{\max}}}$$

它和普通 softmax 等价，因为所有 logits 同时减去 $z_{\max}$ 。

■ 题型 C：argmax 不变

$$\arg \max_k \text{softmax}(z)_k = \arg \max_k z_k$$

因为 exp 单调递增，分母相同。

6. Loss / Gradient Descent / Optimizer / PyTorch Training Loop

6.1 Note 里讲什么

训练就是最小化 loss：

$$\min_{\theta} L(\theta; D)$$

gradient descent：

$$\theta^{t+1} = \theta^t - \eta \nabla L(\theta^t)$$

SGD 使用 mini-batch 近似 full gradient。

6.2 PyTorch 训练循环
<pre>output = model(x) loss = criterion(output, target) optimizer.zero_grad() loss.backward() optimizer.step()</pre> 含义: forward -> 计算 output criterion -> 计算 loss zero_grad -> 清空旧梯度 backward -> 反向传播, 计算梯度 step -> 更新参数

6.3 已考題: MyMLP
代码: nn.Linear(4, 8, bias=True) nn.ReLU() nn.Linear(8, 2, bias=True) 输入: x.shape = (1,4) 输出: (1,4) -> Linear(4,8) -> (1,8) (1,8) -> ReLU -> (1,8) (1,8) -> Linear(8,2) -> (1,2) 所以: $output.shape = (1,2)$
参数数量: 第一层: $8 \times 4 + 8 = 40$
第二层: $2 \times 8 + 2 = 18$
总数: 58

6.4 还可能怎么考代码
■ 题型 A: requires_grad 和 grad - requires_grad=True: 需要 PyTorch 追踪梯度 - grad: 保存 backward 后的梯度 - nn.Parameter: Module 中可学习参数 ■ 题型 B: omit zero_grad 梯度累加: grad = g1 grad = g1 + g2 grad = g1 + g2 + g3 除非故意 gradient accumulation, 否则错误。 ■ 题型 C: omit step 算了梯度, 但不更新参数。 ■ 题型 D: one big step vs many small steps 一般不能用一个更大学习率的一步代替 10 步, 因为: $\nabla L(\theta_0), \nabla L(\theta_1), \dots$
每一步梯度都在不同参数位置重新计算。 ■ 题型 E: Adam / momentum 解释 可能只问概念: - Momentum: 用过去梯度的 moving average 平滑方向 - Adam: 同时维护一阶矩和二阶矩估计 - AdamW: Adam 加 decoupled weight decay

7. Backpropagation / Computation Graph
7.1 Note 里讲什么 深度网络可以写成 computation graph. 求导时从输出反向沿箭头走。应用 chain rule。 如果一个节点通过多条路径影响输出, 则多条路径的梯度贡献相加。

7.2 已考: Data Flow Graph
如果定义: $z_1 = f_1(x)$ $z_2 = f_2(z_1), \quad z_3 = f_3(z_1)$ $z_4 = f_4(z_2, z_3)$ $z_5 = f_5(z_4), \quad z_6 = f_6(z_4)$ $y = f_7(z_5, z_6)$

则:

$$\frac{\partial y}{\partial x} = \left(\frac{\partial f_7}{\partial z_5} \frac{\partial f_5}{\partial z_4} + \frac{\partial f_7}{\partial z_6} \frac{\partial f_6}{\partial z_4} \right) \left(\frac{\partial f_4}{\partial z_2} \frac{\partial f_2}{\partial z_1} + \frac{\partial f_4}{\partial z_3} \frac{\partial f_3}{\partial z_1} \right) \frac{\partial f_1}{\partial x}$$

7.3 还可能怎么考
■ 题型 A: 给你一个 graph , 让你写 forward 函数 按 topological order 定义中间变量。 ■ 题型 B: 给你一个 graph , 让你求某参数梯度 规则: 从 loss 到参数所有路径: 每条路径导数相乘; 多条路径求和。 ■ 题型 C: 解释 autograd PyTorch 自动构建 computation graph, backward() 按链式法则反向传播梯度。

8. Convolution / CNN / Padding / Stride / Dilation / Bias
8.1 Note 里讲什么 1D convolution: $y_i = \sum_{j=1}^p a_j x_{i+j-1}$ 其中: x = input signal , 长度 n a = kernel/filter , 长度 p y = output 无 padding, stride=1 时: $L_{out} = n - p + 1$

8.2 Padding 是什么?
Padding = 在输入两端补值。最常见是 zero padding。 目的: 避免卷积后尺寸变小 控制输出大小 保留边界信息 ■ 1D 标准输出长度公式 如果: input length = L kernel size = K padding = P stride = S dilation = D 则:

$$L_{out} = \left\lfloor \frac{L + 2P - D(K-1) - 1}{S} + 1 \right\rfloor$$

无 dilation 时 $D=1$:

$$L_{out} = \left\lfloor \frac{L + 2P - K}{S} + 1 \right\rfloor$$

8.3 valid / same padding
■ valid convolution 无 padding: $P = 0$
输出会变小: $L_{out} = L - K + 1$

stride=1. ■ same convolution 希望输出长度和输入一样, stride=1 时常取: $P = \frac{K-1}{2}$
如果 K 是奇数。 例如: L=10, K=3, P=1, S=1 $L_{out} = 10 + 2 - 3 + 1 = 10$

8.4 Stride
Stride = kernel 每次移动几步。 1D 公式: $y_i = \sum_{j=1}^p a_j x_{s(i-1)+j}$

stride 越大, 输出越短。
 例子:
 L=10, K=3, P=0, S=2

$$L_{out} = \left\lfloor \frac{10 - 3}{2} + 1 \right\rfloor = \lfloor 4.5 \rfloor = 4$$

8.5 Dilation
Dilation = kernel 内部采样间隔。 有效 kernel size: $K_{eff} = D(K-1) + 1$

输出: $L_{out} = \left\lfloor \frac{L + 2P - K_{eff}}{S} + 1 \right\rfloor$
例子: K=3, D=2 有效覆盖长度: $2(3-1) + 1 = 5$
相当于 kernel 中间插空。

8.6 Bias
卷积层通常: $y = a * x + b$
bias 是每个 output channel 一个标量。

8.7 2D CNN shape
输入: (C_{in}, H, W)
卷积层: nn.Conv2d(C_in, C_out, kernel_size=K, padding=P, stride=S) 输出: $(C_{out}, H_{out}, W_{out})$
其中: $H_{out} = \left\lfloor \frac{H + 2P - K}{S} + 1 \right\rfloor$ $W_{out} = \left\lfloor \frac{W + 2P - K}{S} + 1 \right\rfloor$
如果 kernel 是 $K_h \times K_w$, padding/stride 也分别算。

8.8 Conv2d 参数量
$C_{out} \times C_{in} \times K_h \times K_w + C_{out}$
例子: nn.Conv2d(3, 16, kernel_size=3, bias=True) 参数量: $16 \times 3 \times 3 \times 3 + 16 = 448$
8.9 已考关联: 1D deconvolution / transposed convolution

这其实是 convolution 的逆向尺寸操作, 但不是严格数学 inverse。
 无 padding、无 dilation、无 output padding 时:

$L_{out} = (L_{in} - 1)S + K$
你之前题目: $L_{in} = 8, \quad S = 2, \quad K = 3$
所以: $L_{out} = (8 - 1) \times 2 + 3 = 17$
每个输入值乘 kernel, 然后按 stride 放到输出上, 重叠相加。

8.10 Transposed convolution 标准输出公式
如果有 padding/dilation/output_padding: $L_{out} = (L_{in} - 1)S - 2P + D(K - 1) + output_padding + 1$
无 dilation $D=1$: $L_{out} = (L_{in} - 1)S - 2P + K + output_padding$
■ 例子 L_in=8, K=3, S=2, P=1, output_padding=0 $L_{out} = (8 - 1)2 - 2 + 3 = 15$
如果 output_padding=1: $L_{out} = 16$

8.11 同一部分还可能考什么
■ 可能题 A: 给 input/kernel/padding/stride 手算 convolution 步骤: 1. 先 padding 输入 2. 按 stride 移动 kernel 3. 每个位置做 dot product 4. 加 bias ■ 可能题 B: 问 padding 怎么办 一定先声明 assumption: Assume zero padding with P zeros on each side. 然后再算输出。 ■ 可能题 C: 问 output shape 直接套公式。 ■ 可能题 D: 问 Conv 参数量 套: $C_{out} C_{in} K_h K_w + C_{out}$
■ 可能题 E: 问 transposed conv 和 upsampling 区别 upsampling usually has no learnable parameters transposed convolution has learnable kernel

9. Pooling / Upsampling / Unpooling
9.1 Pooling Average pooling: 窗口内取平均 Max pooling: 窗口内取最大 常见: 2×2 window, stride=2 会让 H,W 各减半, 总像素变成四分之一。
9.2 Pooling 可能考什么
■ 题型 A: 手算 max pooling 输入: $\begin{bmatrix} 1 & 3 \\ 2 & 0 \end{bmatrix}$
max pooling 输出: 3

■ 题型 B: backprop through pooling Average pooling: 如果 2×2 平均池化, 上游梯度是 g , 则每个输入拿: $g/4$
Max pooling: 上游梯度只传给 argmax 位置, 其他位置是 0。
9.3 Upsampling
Upsampling = 把信号放大。 它不一定有 learnable parameters。 ■ Nearest neighbour 每个值复制。 例如三倍: $(6, 1) \rightarrow (6, 6, 6, 1, 1, 1)$

■ Linear interpolation 在相邻点之间插入等间隔值。 从 u 到 v , 三倍时写: $u, \quad u + \frac{v-u}{3}, \quad u + \frac{2(v-u)}{3}$

9.4 Padding / boundary 怎么办?
Upsampling 的边界处理很容易有不同答案, 所以考试要写 assumption。 常见边界 assumption: ■ repeat boundary 最后一个值重复: ..., 2, 2, 2 ■ align corners 把第一个和最后一个原始点对齐到新序列的两端, 再按坐标插值。 这种结果和“每段插两个值然后重复最后值”可能不同。 ■ zero padding boundary 边界外当成 0。 如果题目说 padding, 可能是指插值时超出边界用 0; 必须按题目定义。 考试策略: 题目没说清楚时, 一定写: Assume we insert two equally spaced values between adjacent samples and repeat the final value to reach length 3n. 如果题目明确说 align_corners=True/False, 按 PyTorch 规则算。 ■ 一个具体计算例子 设输入只有两个点: $x = (2, 8)$
我们想做三倍 upsampling, 所以输出长度是: $2 \times 3 = 6$

情况 1: repeat boundary 先在 2 和 8 之间插两个等距点:
$2, \quad 2 + \frac{8-2}{3} = 4, \quad 2 + \frac{2(8-2)}{3} = 6, \quad 8$
这时只有 4 个值, 还差 2 个值。 按 repeat boundary, 把最后一个值重复: $(2, 4, 6, 8, 8)$

情况 2: align corners 这时不是“每段插两个值然后重复最后值”, 而是把整个区间两端对齐。输出有 6 个点, 所以它们在区间 [0, 1] 上的坐标是:
$0, \frac{1}{5}, \frac{2}{5}, \frac{3}{5}, \frac{4}{5}, 1$
原始左端点是 2, 右端点是 8, 所以每个位置的值都是: $2 + (8-2)t = 2 + 6t$
代入各个 t : $\left(2, 2 + \frac{6}{5}, 2 + \frac{12}{5}, 2 + \frac{18}{5}, 2 + \frac{24}{5}, 8 \right) = \left(2, \frac{16}{5}, \frac{22}{5}, \frac{28}{5}, \frac{34}{5}, 8 \right)$

也就是: $(2, 3.2, 4.4, 5.6, 6.8, 8)$
注意它和 repeat boundary 的 $(2, 4, 6, 8, 8)$
明显不同。

情况 3: zero padding boundary
如果题目明确说“边界外当成 0”, 那你可以把最后一个点右边看成接了一个 0。 前 4 个值还是来自 2 到 8 这一段: $2, 4, 6, 8$
最后两个值来自 8 到 0 这一段: $8 + \frac{0-8}{3} = \frac{16}{3}, \quad 8 + \frac{2(0-8)}{3} = \frac{8}{3}$
所以这一种可能得到: $\left(2, 4, 6, 8, \frac{16}{3}, \frac{8}{3} \right)$
也就是: $(2, 4, 6, 8, 5.\bar{3}, 2.\bar{6})$
但是这一种只能在题目明确说“边界外接 0 处理”时使用。

所以这个知识点的核心不是死记某一个答案，而是：先看题目到底采用哪一种 boundary assumption。如果题目没说，就主动写出 assumption 再开始算。

9.5 已考 upsampling 题答案

输入：
$$x = (6, 1, 2, 5, 8, 6, 4, 2)$$

nearest neighbour 三倍：
$$(6, 6, 6, 1, 1, 1, 2, 2, 2, 5, 5, 8, 8, 8, 6, 6, 6, 4, 4, 2, 2, 2)$$

linear interpolation，使用“每段插两个值，最后重复补齐”：

$$\left(6, \frac{13}{3}, \frac{8}{3}, 1, \frac{4}{3}, \frac{5}{3}, 2, 3, 4, 5, 6, 7, 8, \frac{22}{3}, \frac{20}{3}, 6, \frac{16}{3}, \frac{14}{3}, 4, \frac{10}{3}, \frac{8}{3}, 2, 2, 2\right)$$

10. RNN / Sequence Models / BPTT

10.1 Note 里讲什么

RNN 处理序列：
$$h_t = f(x_t, h_{t-1})$$

输出：
$$y_t = g(x_t, h_{t-1})$$

hidden state 是记忆。

10.2 已考 linear RNN

给：
$$y_t = Ax_t + Bh_{t-1} + c$$

$$h_t = Dx_t + Eh_{t-1} + f$$

$$h_0 = 0$$

说明是 RNN：
因为 h_t 依赖 h_{t-1} ，所以有 recurrence。
展开：

$$h_1 = Dx_1 + f$$
$$h_2 = Dx_2 + EDx_1 + Ef + f$$

$$h_3 = Dx_3 + EDx_2 + E^2Dx_1 + E^2f + Ef + f$$

一般：

$$h_{t-1} = \sum_{i=1}^{t-1} E^{t-1-i}Dx_i + \sum_{i=0}^{t-2} E^i f$$

代入：

$$y_t = Ax_t + \sum_{i=1}^{t-1} BE^{t-1-i}Dx_i + B \sum_{i=0}^{t-2} E^i f + c$$

所以 y_t 是 $x_1, \dots, x_t$ 的 affine function。

10.3 还可能怎么考

■ 题型 A：给 RNN 参数 shape，判断输出 shape 如果：

$$x_t \in \mathbb{R}^n$$

$$h_t \in \mathbb{R}^h$$

$$y_t \in \mathbb{R}^m$$

则：

$$D \in \mathbb{R}^{h \times n}$$

$$E \in \mathbb{R}^{h \times h}$$

$$A \in \mathbb{R}^{m \times n}$$

$$B \in \mathbb{R}^{m \times h}$$

■ 题型 B：BPTT Backpropagation through time = 把 RNN 沿时间展开成很深的 feedforward graph，然后反向传播。

■ 题型 C：vanishing/exploding gradients 因为反复乘：

$$E, E^2, E^3, \dots$$

如果 eigenvalues 太小，梯度消失；太大，梯度爆炸。

11. Embeddings / Language Models / Token Prediction

11.1 Note 里讲什么

词可以被 one-hot 表示，也可以被 embedding matrix 映射到 dense vector。
如果 vocabulary size 是 $|V|$ ，embedding dimension 是 d ，embedding 参数量：

$$|V| \times d$$

one-hot 输入乘 embedding matrix，本质是取出对应行/列。

11.2 可能考什么

■ 题型 A：embedding 参数量
nn.Embedding(num_embeddings=10000, embedding_dim=512)
参数量：

$$10000 \times 512 = 5,120,000$$

■ 题型 B：language model loss 预测 next token 通常用 cross entropy：
logits shape = (B, T, vocab_size)
target shape = (B, T)
每个位置预测下一个 token。

■ 题型 C：causal mask 自回归模型不能看未来 token，所以 attention logits 中未来位置设为：

$$-\infty$$

softmax 后未来权重为 0。

12. Transformer / Attention / Positional Encoding / KV Cache / Flash Attention

12.1 Attention 公式

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

其中：

$$Q \in \mathbb{R}^{m \times d}$$

$$K \in \mathbb{R}^{n \times d}$$

$$V \in \mathbb{R}^{n \times p}$$

所以：

$$QK^\top \in \mathbb{R}^{m \times n}$$

$$\text{softmax}(\cdot) \in \mathbb{R}^{m \times n}$$

$$\text{output} \in \mathbb{R}^{m \times p}$$

12.2 已考 single-query attention

你给的题：

$$q = [1, 0, 1]^\top$$

$$K = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & -1 \end{bmatrix}$$

$$V = \begin{bmatrix} 7 & 1 & 3 \\ 4 & 2 & 6 \end{bmatrix}$$

算：

$$q^\top K^\top = [2, 0]$$

因为 $d = 3$ ：

softmax：

$$\frac{q^\top K^\top}{\sqrt{d}} = \left[\frac{2}{\sqrt{3}}, 0\right]$$

乘 V ：

$$z = \frac{1}{e^{2/\sqrt{3}} + 1} [7e^{2/\sqrt{3}} + 4, e^{2/\sqrt{3}} + 2, 3e^{2/\sqrt{3}} + 6]$$

12.3 还可能怎么考

■ 题型 A：multi-query attention shape 如果：

$$Q \in \mathbb{R}^{4 \times 8}$$

$$K \in \mathbb{R}^{6 \times 8}$$

$$V \in \mathbb{R}^{6 \times 10}$$

则：

$$QK^\top \in \mathbb{R}^{4 \times 6}$$

softmax 后：

$$4 \times 6$$

乘 V ：

$$(4 \times 6)(6 \times 10) = 4 \times 10$$

■ 题型 B：为什么除以 $\sqrt{d}$ 如果 d 很大，dot product 方差会变大，softmax 可能过度尖锐。除以 $\sqrt{d}$ 保持 logits 尺度更稳定。
■ 题型 C：positional encoding Transformer 自身没有顺序概念：positional encoding 给 token 加位置信息。否则 transformer 对输入 token 顺序 permutation equivariant。

■ 题型 D：multi-head attention 把 embedding 分成多个 head，各自做 attention，然后 concat，再 linear projection。
常考概念：
multi-head allows attending to different relations/subspaces

■ 题型 E：KV cache 自回归生成时，之前 token 的 key/value 可以缓存。
新 token 只需要算当前 query 和缓存的 keys/values。query 不需要缓存，因为未来不会用旧 query。

■ 题型 F：Flash Attention / online softmax
Flash Attention 的核心之一是减少内存读写，并用 online softmax 分块计算 attention，而不用完整存下巨大 attention matrix。

13. Online Softmax 证明

13.1 Stable softmax

$$\text{softmax}(z_i) = \frac{e^{z_i - z_{\max}}}{Z}$$

$$Z = \sum_i e^{z_i - z_{\max}}$$

13.2 已考证明：loop invariant

算法一遍扫描维护：

$$z_{\max}$$

$$Z$$

和：

循环不变量：
处理完前 i 个数后：

$$z_{\max}^{(i)} = \max(z_1, \dots, z_i)$$

$$Z^{(i)} = \sum_{j=1}^i e^{z_j - z_{\max}^{(i)}}$$

■ Case 1：新元素不是最大 如果：

$$z_i \leq m$$

$$Z \leftarrow Z + e^{z_i - m}$$

成立。
■ Case 2：新元素是最大 如果：

$$z_i > m$$

旧：

$$Z = \sum_{j=1}^{i-1} e^{z_j - m}$$

新 reference 是 z_i ，所以旧项要乘：

$$e^{m - z_i}$$

因为：

$$e^{z_j - z_i} = e^{z_j - m} e^{m - z_i}$$

$$e^{z_i - z_i} = 1$$

所以 invariant 成立。

13.3 还可能怎么考

■ 题型 A：给具体 vector 手算 online update 例如：

$$z = [1, 3, 2]$$

初始化：

$$m = -\infty, Z = 0$$

读 1：

$$m = 1, Z = 1$$

读 3，新 max：

$$Z = 1 \cdot e^{1-3} = e^{-2}$$

更新 $m = 3$ ，再加当前：

$$Z = e^{-2} + 1$$

读 2，不是新 max：

$$Z = e^{-2} + 1 + e^{2-3} = e^{-2} + 1 + e^{-1}$$

最终：

$$m = 3$$

$$Z = e^{1-3} + e^{3-3} + e^{2-3}$$

正确。

14. Numerical Issues / Debugging / Diagnostics

14.1 Note 里讲什么

深度学习数值计算会遇到：

- overflow / underflow
- finite precision
- NaN / Inf
- 梯度爆炸/消失
- loss 不下降
- train/test gap

14.2 可能考什么

■ 题型 A：为什么 stable softmax 普通：

$$e^{1000}$$

可能 overflow。
stable softmax 减去 max：

$$e^{z_i - z_{\max}}$$

最大项变成 $e^0 = 1$ ，更稳定。

■ 题型 B：训练 debugging 如果 training loss 不下降，可能原因：
learning rate 太大/太小
忘记 optimizer.step()
忘记 loss.backward()
模型输出 shape 不对
target 格式不对
梯度被 detach
参数 requires_grad=False
如果 validation loss 高但 training loss 低：overfitting
解决：
regularization
data augmentation
dropout
early stopping
smaller model

15. Normalization / Dropout / Data Augmentation

15.1 可能考概念

■ BatchNorm 按 batch 统计均值方差，标准化 activation。
训练和测试行为不同：训练用 batch statistics，测试用 running statistics。

■ LayerNorm 对单个样本内部 feature 维度做 normalization。
Transformer 常用。

■ Dropout 训练时随机把部分 activation 置零，测试时不用 dropout。
用于 regularization。

■ Data augmentation 对输入做变换，增加数据多样性。
图像中常见：

- crop
 - flip
 - color jitter
 - blur
 - grayscale
- 你项目里的 ablation 就是这部分：去掉某个 augmentation，看 downstream performance 变化。

16. Autoencoder / VAE / ELBO

16.1 Autoencoder

Encoder：

$$z = E(x)$$

$$z = E(x)$$

Decoder：

$$\hat{x} = D(z)$$

$$\hat{x} = D(z)$$

$$\hat{x} \approx x$$

$$\hat{x} \approx x$$

$$\hat{x} \approx x$$

latent z 是 bottleneck。

16.2 VAE

VAE 不是输出一个确定 z ，而是输出 latent distribution：

$$q(z | x)$$

$$q(z | x)$$

再从中 sample：

$$z \sim q(z | x)$$

$$z \sim q(z | x)$$

然后 decoder 生成：

$$p(x | z)$$

$$p(x | z)$$

16.3 已考 ELBO 推导
目标:
$\log p(x) = \log \int p(x, z) dz$
引入:
$q(z \mid x)$
乘 q / q :
$\log p(x) = \log \int q(z \mid x) \frac{p(x, z)}{q(z \mid x)} dz$
用 Jensen:
$\log E_q[f(z)] \geq E_q[\log f(z)]$
得到:
$\log p(x) \geq E_q \left[\log \frac{p(x, z)}{q(z \mid x)} \right]$
拆:
$p(x, z) = p(x \mid z) p(z)$
$= E_q[\log p(x \mid z)] - E_q \left[\log \frac{q(z \mid x)}{p(z)} \right]$
所以:
$ELBO = E_q[\log p(x \mid z)] - KL(q(z \mid x) \parallel p(z))$

16.4 还可能怎么考
■ 题型 A: 解释两项含义
$E_q[\log p(x \mid z)]$
是 reconstruction term。
$KL(q(z \mid x) \parallel p(z))$
是 regularization term, 让 latent posterior 靠近 prior。 ■ 题型 B: reparameterization trick 如果:
$z \sim N(\mu, \Sigma)$
可以写成:
$z = \mu + \Sigma^{1/2} \epsilon$
其中:
$\epsilon \sim N(0, I)$
这样可以让 sampling 过程可微, 方便 backprop。 如果 diagonal Gaussian:
$z = \mu + \sigma \odot \epsilon$

17. Diffusion / DDPM / DDIM
17.1 Forward diffusion
逐渐给图像加噪声:
$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$
其中:
$\epsilon \sim N(0, I)$
模型学习预测噪声:
$\epsilon_{\theta}(x_t, t)$
17.2 训练目标
常见 diffusion loss:
$\ \epsilon - \epsilon_{\theta}(x_t, t) \ ^2$
也就是让网络预测加进去的噪声。

17.3 已考 DDIM
给:
$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} x_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \epsilon_{\theta}(x_t, t) + \sigma_t z$
令:
$\sigma_t = 0$
得:
$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} x_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \epsilon_{\theta}(x_t, t)$
没有随机项, 所以 deterministic。
17.4 代入 x_0 estimate
由 forward 公式:
$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$
得到:
$x_0 \approx \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}(x_t, t)}{\sqrt{\bar{\alpha}_t}}$
代入 deterministic DDIM:
$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}(x_t, t)}{\sqrt{\bar{\alpha}_t}} + \sqrt{1 - \bar{\alpha}_{t-1}} \epsilon_{\theta}(x_t, t)$

17.5 还可能怎么考
■ 题型 A: 解释 DDPM vs DDIM DDPM: reverse step includes random noise sampling stochastic DDIM: can set sigma_t=0 sampling deterministic can use fewer steps ■ 题型 B: ODE analogy 当采样确定时, 路径不再随机, 可以解释为沿一个确定性 vector field 求解 ODE。 ■ 题型 C: 给 $x_t, a, \epsilon_{\theta}$ 手算 x_0 直接套:
$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}}{\sqrt{\bar{\alpha}_t}}$

18. Contrastive Learning / SimCLR / CLIP
18.1 Note 里讲什么
Representation learning 目标是学习有用 embedding。 Contrastive learning 的核心: positive pairs 拉近 negative pairs 推远
18.2 InfoNCE / softmax 型 loss
一般形式:
$\ell = -\log \frac{\exp(\text{sim}(z_i, z_j) / \tau)}{\sum_k \exp(\text{sim}(z_i, z_k) / \tau)}$
和 cross entropy / softmax 非常像。

18.3 可能考什么
■ 题型 A: 解释 temperature τ
控制 softmax sharpness: - 小 τ: 更强调最相似样本 - 大 τ: 分布更平
■ 题型 B: collapse 如果所有输入都映射到同一个 embedding, 就是 collapse。 避免方法: negative samples feature normalization strong augmentation projection head teacher-student EMA
■ 题型 C: CLIP 图像 encoder 和文本 encoder 映射到同一 embedding space。 正确 image-text pair 相似度高, 错误 pair 相似度低。

19. Reinforcement Learning / Policy Gradient / Actor-Critic
19.1 这部分如果考, 通常偏概念 + 公式
关键词: state s action a reward r policy $\pi(a s)$ value $V\pi(s)$ Q-value $Q\pi(s,a)$ advantage $A\pi(s,a)=Q\pi(s,a)-V\pi(s)$
19.2 可能考什么
■ 题型 A: advantage 含义
$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$
表示 action a 比当前 policy 在 state s 的平均表现好多少。 ■ 题型 B: actor-critic - actor: 学习 policy - critic: 估计 value / advantage
■ 题型 C: policy gradient 直觉 提高高回报 action 的概率, 降低低回报 action 的概率。

21. 超短速查版
核心公式
■ Linear
$y = Wx + b, \quad \#params = out \cdot in + out$
■ Softmax
$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}, \quad softmax(z) = softmax(z + c1)$
■ CrossEntropyLoss
$L = -\log(softmax(logits)[target])$
input = logits, target = class index
■ Backprop
路径导数相乘, 多路径相加
■ Conv output
$\left\lfloor \frac{L + 2P - D(K - 1) - 1}{S} + 1 \right\rfloor$

■ Conv2d params
$C_{out} C_{in} K_h K_w + C_{out}$
■ Transposed conv output
$(L_{in} - 1)S - 2P + D(K - 1) + output_padding + 1$
■ Pooling
avg: 梯度平均分 max: 梯度给 argmax
■ RNN
展开 h_t , 反复出现 E^k
■ Attention
$QK^T / \sqrt{d} \rightarrow row\ softmax \rightarrow V$
■ Online softmax
新 max 出现时, 旧 Z 乘 $\exp(old_max - new_max)$
■ ELBO
$\log p(x) \geq E_q[\log p(x \mid z)] - KL(q(z \mid x) \parallel p(z))$
■ DDIM
$\sigma = 0 \rightarrow \text{deterministic}$
$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}}{\sqrt{\bar{\alpha}_t}}$